Guide 1 · Concepts

Every idea AgriAgent rests on, explained from zero

AgriAgent Final Year Project · 11 October 2026

> **How to read this.** Each section is one idea. After each, close the document and explain the idea in two or three sentences. The self-check questions at the end are close to what examiners ask.

1 The agricultural problem

Weeds compete with crops for water, light and nutrients. Farmers control them with **herbicides** (weed-killing chemicals). The usual method is **broadcast spraying**: covering the entire field evenly. That wastes chemical on bare soil and on the crop, costs money, and pollutes soil and water.

Spot spraying (also called site-specific or selective spraying) sprays only where weeds are. Commercial systems exist (for example John Deere See & Spray), but they rely on detectors trained on large labelled datasets for specific crops.

AgriAgent's idea: use general-purpose vision models that need **no training on our data** to produce a **spray map**, a picture of which areas should be sprayed. The key quantity is:

$$\text{herbicide savings} = 1 - \frac{\text{area to spray}}{\text{total area}}$$

If weeds (plus a small safety margin) cover 10% of the image, savings are about 90%.

2 Images are just numbers

A colour image of height H and width W is a 3-D array of shape $H \times W \times 3$: for every pixel, three numbers (red, green, blue) from 0 to 255. In the code you will see images as PIL `Image` objects (a Python imaging library) and as NumPy arrays.

A **mask** is a 2-D array of shape $H \times W$ with `True` where the object is and `False` elsewhere. A spray map is just a mask: `True` means "spray this pixel". Counting `True` pixels gives the area.

3 Three vision tasks

Task	Question it answers	Output
Classification	What is in this image?	one label
Detection	What objects are where?	boxes + labels + scores
Segmentation	Exactly which pixels belong to each object?	masks

A **bounding box** is four numbers (x_1, y_1, x_2, y_2): the top-left and bottom-right corners, in pixels. Each box comes with a **label** ("weed") and a **confidence score** between 0 and 1. A box is coarse: it always contains soil and neighbouring plants in its corners. That is why we need segmentation.

Instance segmentation gives a separate mask per object (weed 1, weed 2, …). That is what AgriAgent produces.

4 Closed versus open vocabulary, and zero-shot

A traditional detector such as YOLO is **closed-vocabulary**: it is trained on a fixed list of classes and can only output those. To detect "pigweed" you must collect and label hundreds of pigweed images and train it.

An **open-vocabulary** detector takes a text description at run time and finds whatever matches. **Zero-shot** means it works on categories it was never specifically trained for. This is AgriAgent's main argument: no labelled training data, any crop, any weed name, just change the prompt.

The trade-off you must be able to state: a zero-shot model is usually **less accurate** on a specific domain than a model trained on that domain. It also only knows what words mean in its general training data. "Weed" is not a visual category: a weed is just a plant growing where the farmer does not want it. This is exactly why our real run failed (see the Week 2 dossier).

5 Grounding DINO

Grounding DINO (IDEA Research, 2023) is an open-vocabulary detector. We use the **tiny** version (`IDEA-Research/grounding-dino-tiny`, about 172 million parameters).

How it works, at the level you need:

1. An **image encoder** (a Swin Transformer) turns the image into feature vectors.
2. A **text encoder** (BERT) turns the prompt into feature vectors, one per word piece.
3. A **fusion** step lets image and text features attend to each other ("cross-modality attention"), so image regions that match words are strengthened.
4. A decoder outputs boxes. Each box gets a score for how well it matches each phrase in the prompt.

Prompt format. Phrases are separated by full stops: `"weed . grass . pigweed ."`. The label of each box is the phrase it matched best. **The model does not "know" a weed from a crop; it matches words to image regions.**

Two thresholds (in `src/vision/grounding_dino.py`): `box_threshold = 0.35` drops boxes with a lower score, and `text_threshold = 0.25` decides which words are attached to a box as its label.

6 SAM 2

SAM 2 (Segment Anything Model 2, Meta AI, 2024) is a **promptable segmentation** model. You give it an image plus a **prompt**, a box or a few clicked points, and it returns the mask of the object you meant. We use `facebook/sam2-hiera-tiny` (about 39 million parameters).

1. An **image encoder** (Hiera) computes image features once.
2. A **prompt encoder** turns boxes and points into vectors. Points can be **positive** ("this is part of the object") or **negative** ("this is not").
3. A **mask decoder** combines both and outputs a mask plus a **predicted IoU**: the model's own guess of how good the mask is.

SAM 2 also has a memory for video; AgriAgent only uses single images.

Important: predicted IoU is the model's self-estimate, not real accuracy. Real accuracy needs a human-drawn ground-truth mask to compare against.

7 Measuring masks: IoU and Dice

Compare a predicted mask P with a ground-truth mask G (drawn by a person):

$$\text{IoU} = \frac{|P \cap G|}{|P \cup G|} \qquad \text{Dice} = \frac{2\,|P \cap G|}{|P| + |G|}$$

$|P \cap G|$ is the number of pixels in both; $|P \cup G|$ the number in either. Both scores range from 0 (no overlap) to 1 (perfect). Example: prediction 100 px, truth 100 px, overlap 50 px: IoU $= \frac{50}{150} = 0.33$, Dice $= \frac{100}{200} = 0.5$. Dice is always at least as large as IoU.

For detection we also use **precision** (of the weeds we found, what fraction are really weeds?) and **recall** (of all real weeds, what fraction did we find?). For our project the most important number is a safety one: **what fraction of crop pixels ended up on the spray map?** We want that near zero.

8 Refinement: asking again with a hint

If SAM 2's predicted IoU for a mask is below 0.85, AgriAgent tries to improve it. It computes an **error centroid**, a point where the mask probably missed foliage, and asks SAM 2 again with that point as a positive click. This repeats at most twice. The idea comes from MedSAM-Agent (medical imaging). The code uses **morphology**: simple operations on masks such as dilation (grow by a few pixels), erosion (shrink), opening and closing (remove specks, fill holes).

9 Deterministic rules versus AI models

A **deterministic** function always gives the same output for the same input and can be read line by line: `if infestation > 85: suppress`. A neural network is **probabilistic**: it can be confidently wrong, and you cannot list everything it will do.

For a physical action such as spraying chemical, AgriAgent uses deterministic rules as the final authority. Two related terms:

- **Fail-closed** (or fail-safe): when something is wrong or unknown, choose the safe action, which here means **do not spray**.
- **Human in the loop**: a person reviews the output before anything real happens. AgriAgent's map is advisory.

10 System 1 and System 2

Borrowed from psychology (Daniel Kahneman, *Thinking, Fast and Slow*). **System 1** is fast and automatic; **System 2** is slow and deliberate. In AgriAgent, System 1 is the vision pipeline plus rules, used for every image. System 2 is the slower reasoning step (a language model plus a document lookup), used only when System 1 is unsure. Use this as an analogy, not as a claim about the brain.

11 LLMs, VLMs and structured output

A **large language model** (LLM) predicts text. A **vision-language model** (VLM) also accepts images. AgriAgent asks a VLM to look at the photo and reply in **JSON** (a fixed machine-readable format) with the crop, the weed, the growth stage and the severity. The reply is checked against a schema (Pydantic), so malformed answers are rejected rather than silently used.

Hallucination means a model inventing confident but false content. A model asked "which herbicide?" might name a chemical that is banned or give a wrong dose. That is why AgriAgent never lets the model choose the chemical; see the next section.

Our models: Groq `qwen/qwen3.8-27b` in the cloud and `Qwen/Qwen3.5-4B` locally. Groq is a company that hosts models on fast hardware; we call it over the internet with an API key.

12 RAG: retrieval-augmented generation

Instead of trusting the model's memory, we **retrieve** the relevant passage from a trusted document and base the answer on it.

1. The trusted document is split into chunks. Ours is the ICAR-DWR herbicide bulletin (`data/knowledge_base/`) plus tables transcribed from it.
2. Each chunk is turned into an **embedding**, a vector of numbers (384 here) such that similar meanings give nearby vectors. The production embedder is `BAAI/bge-small-en-v1.5`; tests use a simple hash-based embedder that works offline.
3. Vectors are stored in a **vector database** (ChromaDB), or in memory for tests.
4. At query time ("herbicide for Trianthema in cotton at vegetative stage"), the query is embedded and the nearest chunks are returned (cosine distance).

AgriAgent goes one step further than usual RAG: the chemical and dose come from the retrieved table row itself, then pass through hard-coded checks (banned list, dose ceiling). The model only names the weed.

13 Indian statutory context

- **CIBRC**: the Central Insecticides Board and Registration Committee, which registers pesticides in India under the Insecticides Act, 1968.
- **ICAR-DWR**: the Directorate of Weed Research. Our knowledge base is their herbicide-use bulletin.
- Doses are in **g a.i./ha**: grams of active ingredient per hectare. **PHI** or waiting period is how long to wait after spraying before harvest.
- Labels carry a **toxicity colour band** (red, yellow, blue, green; red is most toxic).

14 Mock mode, tests and CI

Mock mode replaces each model with a stand-in that returns fixed, plausible output. The pipeline plumbing can then be tested on a laptop with no GPU, quickly and repeatably. Mock output says **nothing** about accuracy: the mock boxes are hard-coded positions.

Unit tests check one function; **end-to-end (E2E) tests** run the whole pipeline. **CI** (continuous integration, GitHub Actions here) runs all checks on every push. Ours runs lint (style), formatting, type checking (mypy), 175 unit tests, 35 E2E tests and an app start-up check, on Python 3.10 and 3.11.

15 GeoJSON

GeoJSON is a standard JSON format for map shapes. A **FeatureCollection** holds **features**; each has a **geometry** (here a **Polygon**: a closed ring of longitude/latitude points) and **properties** (label, dose, …). AgriAgent converts each approved weed mask into polygons. **The coordinates are fake**: pixel positions scaled onto a fixed point near Mumbai. Real coordinates would need GPS and camera calibration.

16 Hardware basics

A model's weights must fit in GPU memory (VRAM). Roughly, memory ≈ parameters × bytes per parameter. At 16-bit precision that is 2 bytes each: a 9-billion-parameter model needs about 18 GB, and a 4-billion one about 8 GB, plus working memory. **Quantization** stores weights in fewer bits (4-bit ≈ 0.5 byte), shrinking memory at a small accuracy cost. Our laptop GPU has 8 GB. So Grounding DINO-tiny and SAM 2-tiny fit easily, Qwen3.5-4B needs 4-bit, and Clef Flash (9B) does not fit.

17 Self-check

1. Why is a bounding box not enough to spray safely?
2. What does "zero-shot" mean, and what is its main weakness for our problem?
3. What is the label of a Grounding DINO box, and where does it come from?
4. What does SAM 2 need as input besides the image?
5. What is the difference between SAM 2's predicted IoU and real IoU?

6. Compute IoU and Dice for prediction 200 px, truth 100 px, overlap 100 px.
7. Why are deterministic rules the final authority for spraying?
8. Explain RAG to a non-technical friend in three sentences.
9. In AgriAgent, does the language model choose the herbicide? What does?
10. Why does mock-mode output say nothing about accuracy?
11. Why can’t Clef Flash 9B run on our laptop?